Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Домашнее задание

«Анализатор оценок на языке Rust»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Сердюков А.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и постановка задачи

Изучить язык Rust на примере небольшого консольного проекта «Анализатор оценок». Программа читает текстовый файл, проверяет записи, вычисляет средние баллы, строит рейтинг студентов и распределение оценок. Проект демонстрирует особенности языка в работающем приложении с обработкой ошибок и автоматическими тестами.

В основном условии ДЗ предлагается выбрать ранее не изучавшийся язык и создать проект с подробным описанием и комментариями. Для этого задания выбран Rust. Номер варианта для основного ДЗ в условии не задан; вариант 22 используется в рубежных контролях.

Входные данные и результат

Вход — UTF-8-файл: в каждой строке фамилия и одна или несколько оценок, разделённые символом ;. Разрешены целые оценки от 2 до 5. Пустые строки и комментарии # пропускаются. Пробелы по краям полей удаляются. Заголовок, кавычки и разделитель внутри поля не поддерживаются; это упрощённый формат, а не полный стандарт CSV.

Программа выводит число записей, рейтинг по убыванию среднего балла и число оценок каждого значения. При равных средних баллах применяется порядок строк Rust по фамилии. Средние сравниваются до округления, а в выводе отображаются с двумя знаками после точки.

Средства разработки

Проверка выполнена в Linux с rustc 1.99.1 и Cargo 1.99.1; установлены компоненты rustfmt и Clippy. Проект можно открыть в Visual Studio Code через WSL. Пакет Cargo содержит библиотеку src/lib.rs и исполняемый модуль src/main.rs. Сторонних зависимостей нет, проект собирается с параметром --offline.

Запуск

```
cd homework
cargo run --offline -- data/students.csv
cargo test --offline
```

Из корня pcpl-labs можно использовать ./homework/cargo-local.sh вместо cargo; скрипт выбирает Cargo из PATH или стандартной установки ~/.cargo/bin/ и переходит в папку проекта.

Конструкции и парадигмы Rust

Владение и заимствование

Student владеет String с фамилией и Vec<u8> с оценками. В parse_students новые значения перемещаются внутрь структуры. Когда владелец выходит из области видимости, принадлежащие ему значения освобождаются. В этом коде управление ресурсами не требует ручного освобождения памяти.

Метод average принимает &self и только читает запись. Функция ranking принимает срез &[Student] и возвращает Vec<&Student>: коллекция рейтинга содержит ссылки на исходные записи. Время жизни возвращаемых ссылок связано с входным срезом, поэтому результат нельзя использовать после уничтожения исходных данных. Переменная ranked изменяема, а записи Student через эти ссылки не изменяются.

Типы данных и обработка ошибок

Структура struct объединяет поля записи, а блок impl определяет метод вычисления среднего. Перечисление ParseError содержит варианты EmptyInput и InvalidLine с номером строки и причиной. Функция разбора возвращает Result<Vec<Student>, ParseError>. Оператор ? передаёт ошибку вызывающему коду, map_err уточняет описание ошибки, а match в main выбирает код завершения процесса.

Для вывода ошибки реализован trait std::fmt::Display; реализация std::error::Error делает её обычной ошибкой Rust. Атрибуты derive(Debug, PartialEq) дают диагностическое представление и сравнение значений в тестах. unwrap используется только в тестах для заранее корректных примеров, а не для некорректного пользовательского ввода.

Процедурный и функциональный стиль

Чтение файла, цикл разбора и печать реализуют последовательную обработку. Цепочки iter, map, flat_map, fold и collect применяют функциональные приёмы к коллекциям. Замыкание сортировки сравнивает средние баллы и использует then_with для фамилии. BTreeMap сохраняет порядок ключей 2, 3, 4, 5. Таким образом, проект сочетает несколько стилей в рамках системы типов Rust.

Архитектура и алгоритмы

Разбор и валидация

parse_students нумерует исходные строки с единицы, пропускает пустые строки и комментарии, разделяет поля и проверяет фамилию. Повторная фамилия запрещена, поскольку служит ключом учебной записи; регистр при сравнении значим. Далее каждая оценка преобразуется в u8 и проверяется на попадание в диапазон 2..5. Отсутствие оценок, пустое поле, нечисловое значение или выход за диапазон приводят к Err с номером исходной строки.

Ошибочная строка отвергает весь файл: частичный рейтинг не выводится. Файл без записей также возвращает ошибку. Ошибки чтения файла и разбора отображаются в stderr, код завершения равен 1. Для успешного запуска и --help код равен 0.

Вычисления

average суммирует оценки в u32 и делит сумму на количество оценок в f64. Для защитной обработки вручную созданной записи без оценок возвращается 0.0; парсер таких записей не создаёт. ranking собирает ссылки на студентов и сортирует их по убыванию среднего. histogram разворачивает оценки всех записей через flat_map и накапливает счётчики через fold.

Пусть S — число студентов, G — общее число оценок. Проверка уникальности фамилий линейным поиском требует $O(S^2)$, разбор оценок — O(G). В текущей реализации среднее пересчитывается при сравнении элементов рейтинга. При максимуме K оценок на студента сортировка требует $O(S \log S \cdot K)$. Гистограмма имеет четыре фиксированных ключа и строится за O(G). Для небольших учебных данных это приемлемо; для больших данных можно кэшировать средние баллы и хранить фамилии в HashSet.

Файлы проекта

Cargo.toml описывает пакет. src/lib.rs содержит модель, вычисления и модульные тесты; src/main.rs — командный интерфейс. data/students.csv содержит 5 учебных записей и 15 оценок, data/invalid.csv — пример ошибки. Cargo.lock фиксирует пакет, cargo-local.sh упрощает запуск на этом компьютере. Данные составлены для демонстрации и не являются ведомостью группы.

Результаты выполнения

```
$ cargo run --offline -- data/students.csv
Анализ оценок на Rust
Записей: 5
Рейтинг по среднему баллу:
1. Сидорова: 5.00
2. Иванов: 4.67
3. Алексеева: 4.33
4. Петров: 3.33
5. Смирнов: 3.00
Частоты оценок:
2: 1
3: 3
4: 5
5: 6

$ cargo run --offline -- data/invalid.csv
Ошибка: строка 1: оценка вне диапазона 2..5
Код возврата: 1

$ cargo run --offline -- --help
Запуск: grade_analyzer <путь к файлу>
Формат строки: фамилия;оценка;оценка (2..5)
```

Рисунок 1. Экранная форма фактического вывода программы

Изображение построено по сохранённому stdout и stderr. Полные журналы приведены в папке results.

Максимальный средний балл — 5.00 у Сидоровой. Частоты 1 + 3 + 5 + 6 дают 15 оценок, что соответствует пяти входным записям по три оценки. Значение 6 в ошибочном файле отклоняется с указанием строки и кодом завершения 1.

Проверка и выводы

Выполнено шесть модульных тестов. Проверены Unicode и CRLF, средний балл, пустой ввод, ошибочные записи, повтор фамилии с точным номером строки, порядок рейтинга при равенстве средних, сохранность исходного порядка, гистограмма и пустые коллекции. Тесты входят в src/lib.rs и запускаются Cargo.

Шесть проверок командного интерфейса охватывают обычный файл, неверную оценку, отсутствующий файл, отсутствие аргументов, лишние аргументы и --help. Скрипт tools/verify_extra.py проверяет stdout, stderr и коды завершения. Форматирование cargo fmt --check и анализ cargo clippy --all-targets -- -D warnings завершились успешно. Сборка и тесты выполнялись без сторонних зависимостей и без обращения Cargo к сети.

```
test tests::counts_histogram_and_handles_empty_collections ... ok
test tests::parses_unicode_and_crlf ... ok
test tests::rejects_empty_input ... ok
test tests::rejects_invalid_records ... ok
test tests::reports_duplicate_line_number ... ok
test tests::sorts_average_and_name_without_mutation ... ok
test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

Вывод

Реализован работающий проект на Rust с разбором данных, вычислениями и контролем ошибок. На примере проекта изучены владение и заимствование, struct и impl, enum и Result, traits, сопоставление с образцом и итераторы. Выделение библиотеки позволило тестировать вычисления отдельно от чтения файла и печати. Полные тексты проекта приведены далее.

Источники

Условие ДЗ: https://github.com/ugapanyuk/courses_content/wiki/DZ

The Rust Programming Language, гл. 4: владение и заимствование.

https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

Гл. 9.2: Result, передача и обработка ошибок.

https://doc.rust-lang.org/book/ch09-02-recoverable-errors-with-result.html

Гл. 11.1: атрибут test и автоматические проверки.

https://doc.rust-lang.org/book/ch11-01-writing-tests.html

Дата обращения к условиям и документации: 19 октября 2026 года.

Текст программы

```
# Файл Cargo.toml

[package]
name = "grade_analyzer"
version = "0.1.0"
edition = "2021"
description = "Учебный анализатор оценок на Rust для ДЗ по ПиКЯП"

[dependencies]


# Файл src/lib.rs

//! Анализ оценок: владение строками, заимствование, enum и итераторы.
use std::collections::BTreeMap;
use std::fmt;

/// Запись владеет фамилией и вектором оценок: ссылок на входной файл нет.
#[derive(Debug, PartialEq)]
pub struct Student {
    pub name: String,
    pub grades: Vec<u8>,
}

impl Student {
    /// Неизменяемое заимствование позволяет считать среднее без копирования.
    pub fn average(&self) -> f64 {
        let sum: u32 = self.grades.iter().map(|&g| u32::from(g)).sum();
        if self.grades.is_empty() {
            0.0
        } else {
            f64::from(sum) / self.grades.len() as f64
        }
    }
}

/// Ошибка содержит номер строки и причину; panic для входных данных не нужен.
#[derive(Debug, PartialEq)]
pub enum ParseError {
    EmptyInput,
    InvalidLine { line: usize, reason: String },
}

impl fmt::Display for ParseError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::EmptyInput => write!(f, "нет записей об оценках"),
            Self::InvalidLine { line, reason } => {
                write!(f, "строка {line}: {reason}")
            }
        }
    }
}

impl std::error::Error for ParseError {}
```

Текст программы. Продолжение

```
/// Формат: фамилия;оценка;оценка. Заголовок и кавычки не поддерживаются.
/// Пустые строки и строки комментариев, начинающиеся с #, пропускаются.
pub fn parse_students(input: &str) -> Result<Vec<Student>, ParseError> {
    let mut students = Vec::new();
    for (index, raw) in input.lines().enumerate() {
        let line = raw.trim();
        if line.is_empty() || line.starts_with('#') {
            continue;
        }
        let fail = |reason: &str| ParseError::InvalidLine {
            line: index + 1,
            reason: reason.to_owned(),
        };
        let mut fields = line.split(';').map(str::trim);
        let name = fields.next().unwrap_or_default();
        if name.is_empty() {
            return Err(fail("пустая фамилия"));
        }
        if students.iter().any(|s: &Student| s.name == name) {
            return Err(fail("повторная фамилия"));
        }
        let mut grades = Vec::new();
        for field in fields {
            // ? возвращает ошибку вызывающей функции, не прерывая процесс.
            let grade = field
                .parse::<u8>()
                .map_err(|_| fail("оценка должна быть целым числом 2..5"))?;
            if !(2..=5).contains(&grade) {
                return Err(fail("оценка вне диапазона 2..5"));
            }
            grades.push(grade);
        }
        if grades.is_empty() {
            return Err(fail("нет оценок"));
        }
        // Владение созданными String и Vec переносится внутрь Student.
        students.push(Student {
            name: name.to_owned(),
            grades,
        });
    }
    if students.is_empty() {
        Err(ParseError::EmptyInput)
    } else {
        Ok(students)
    }
}

/// Результат содержит ссылки: исходные записи не перемещаются и не
    копируются.
pub fn ranking(students: &[Student]) -> Vec<&Student> {
    let mut ranked: Vec<&Student> = students.iter().collect();
    ranked.sort_by(|a, b| {
        b.average()
            .total_cmp(&a.average())
            .then_with(|| a.name.cmp(&b.name))
```

Текст программы. Продолжение

```
    });
    ranked
}

/// BTreeMap сохраняет порядок ключей. fold накапливает частоты оценок.
pub fn histogram(students: &[Student]) -> BTreeMap<u8, usize> {
    students.iter().flat_map(|s| s.grades.iter()).fold(
        (2..=5).map(|g| (g, 0)).collect(),
        |mut counts, &g| {
            *counts.entry(g).or_default() += 1;
            counts
        },
    )
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn parses_unicode_and_crlf() {
        let data = parse_students("# пример\r\n Иванов ;5;4\r\n").unwrap();
        assert_eq!(data[0].name, "Иванов");
        assert_eq!(data[0].grades, vec![5, 4]);
        assert_eq!(data[0].average(), 4.5);
    }

    #[test]
    fn rejects_empty_input() {
        assert_eq!(parse_students("\n# пусто"), Err(ParseError::EmptyInput));
    }

    #[test]
    fn rejects_invalid_records() {
        for line in [";5", "Анна", "Анна;1", "Анна;6", "Анна;x", "Анна;"] {
            assert!(matches!(
                parse_students(line),
                Err(ParseError::InvalidLine { line: 1, .. })
            ));
        }
    }

    #[test]
    fn reports_duplicate_line_number() {
        assert_eq!(
            parse_students("Анна;5\nАнна;4"),
            Err(ParseError::InvalidLine {
                line: 2,
                reason: "повторная фамилия".to_owned(),
            })
        );
    }

    #[test]
    fn sorts_average_and_name_without_mutation() {
        let data = parse_students("Борис;3\nАнна;5\nВера;5").unwrap();
```

Текст программы. Продолжение

```
        let names: Vec<&str> = ranking(&data).iter().map(|s|
            s.name.as_str()).collect();
        assert_eq!(names, ["Анна", "Вера", "Борис"]);
        assert_eq!(data[0].name, "Борис");
    }

    #[test]
    fn counts_histogram_and_handles_empty_collections() {
        let data = parse_students("Анна;2;4;5\nБорис;4;5").unwrap();
        assert_eq!(
            histogram(&data),
            BTreeMap::from([(2, 1), (3, 0), (4, 2), (5, 2),])
        );
        assert!(ranking(&[]).is_empty());
        assert!(histogram(&[]).values().all(|&n| n == 0));
        assert_eq!(
            Student {
                name: "X".into(),
                grades: vec![]
            }
            .average(),
            0.0
        );
    }
}


# Файл src/main.rs

//! Командная оболочка отделена от функций разбора и вычислений.
use grade_analyzer::{histogram, parse_students, ranking};
use std::{env, fs, process::ExitCode};

fn run() -> Result<(), String> {
    let args: Vec<String> = env::args().skip(1).collect();
    if args == ["--help"] {
        println!("Запуск: grade_analyzer <путь к файлу>");
        println!("Формат строки: фамилия;оценка;оценка (2..5)");
        return Ok(());
    }
    if args.len() != 1 {
        return Err("нужен один путь к файлу; справка: --help".into());
    }
    let input = fs::read_to_string(&args[0])
        .map_err(|e| format!("не удалось прочитать '{}': {e}", args[0]))?;
    let students = parse_students(&input).map_err(|e| e.to_string())?;
    println!("Анализ оценок на Rust");
    println!("Записей: {}", students.len());
    println!("Рейтинг по среднему баллу:");
    for (index, student) in ranking(&students).iter().enumerate() {
        println!("{}. {}: {:.2}", index + 1, student.name, student.average());
    }
    println!("Частоты оценок:");
    for (grade, count) in histogram(&students) {
        println!("{grade}: {count}");
    }
```

Текст программы. Продолжение

```
    Ok(())
}

fn main() -> ExitCode {
    match run() {
        Ok(()) => ExitCode::SUCCESS,
        Err(message) => {
            eprintln!("Ошибка: {message}");
            ExitCode::FAILURE
        }
    }
}


# Файл cargo-local.sh

#!/bin/sh
# Запуск из папки Cargo-проекта независимо от текущего каталога.
set -eu
cd "$(dirname "$0")"
if command -v cargo >/dev/null 2>&1; then
    exec cargo "$@"
fi
if [ -x "$HOME/.cargo/bin/cargo" ]; then
    exec "$HOME/.cargo/bin/cargo" "$@"
fi
printf '%s\n' 'Cargo не найден. Установка:
    https://rust-lang.org/tools/install/' >&2
exit 127
```